

Exercício Prático: API de Agendamento de Petshop

Turma: LionsDev

Tópicos: APIs REST com Express.js, Integração com MongoDB (Mongoose), Schemas e Modelos, Validações básicas com Mongoose, Operações CRUD assíncronas (async/await), Query Params (filtros) e Status Codes HTTP.

1. Contexto

Um petshop chamado **PetLions** deseja modernizar seus processos. Atualmente, o agendamento de serviços de banho e tosa é feito em uma folha de papel, o que costuma causar perda de informações.

Eles precisam de uma API REST simples que conecte a um banco de dados MongoDB (usando Mongoose) para gerenciar esses agendamentos de forma persistente, garantindo que as informações não sejam perdidas quando o servidor reiniciar. A estrutura deve seguir o modelo usado em aula: `server.js`, `db.js` e `models/`, com as rotas escritas diretamente no `server.js`.

2. Configuração Inicial

Crie uma API Node.js com Express e Mongoose.

Requisitos estruturais:

- Crie um arquivo `.env` contendo as variáveis `MONGO_URI` e `PORT` (porta `3000`).
- Crie a pasta `src`.
- Crie o arquivo `src/db.js` para gerenciar a conexão com o MongoDB.
- Crie o arquivo `src/server.js` como ponto de entrada da aplicação.
- Crie a pasta `src/models` e nela o arquivo `agendamento.js`.
- Todas as rotas devem ficar no `src/server.js`, como fizemos em sala.

O Modelo (Schema) do Agendamento

O Schema do Mongoose para os **agendamentos** deve conter os seguintes campos:

- **nomePet** : Tipo **String**, obrigatório.
- **especie** : Tipo **String**, obrigatório (deve aceitar apenas: **Cão**, **Gato** ou **Outro**).
- **nomeDono** : Tipo **String**, obrigatório.
- **telefoneDono** : Tipo **String**, obrigatório.
- **servico** : Tipo **String**, obrigatório (deve aceitar apenas: **Banho**, **Tosa** ou **Banho e Tosa**).
- **data** : Tipo **String**, obrigatório (ex: **"2026-06-15"**).
- **valor** : Tipo **Number** (será calculado de forma automática pela API).
- **status** : Tipo **String**, com valor padrão de **"Agendado"** (deve aceitar apenas: **Agendado**, **Concluído** ou **Cancelado**).

3. Requisitos Obrigatórios e Regras de Negócio

3.1 Cadastrar Agendamento (CREATE)

Crie a rota **POST /agendamentos**.

O corpo da requisição enviará os dados do agendamento (exceto o **valor** e o **status**).

```
{
  "nomePet": "Frederico",
  "especie": "Cão",
  "nomeDono": "Ana Paula",
  "telefoneDono": "(11) 99999-9999",
  "servico": "Banho e Tosa",
  "data": "2026-06-15"
}
```

Regras:

1. **Cálculo Automático do Valor:** Antes de salvar no banco, o backend deve preencher o campo **valor** seguindo esta lógica simples baseada na espécie:
 - Se for **Cão**: Banho (R\$ 50) | Tosa (R\$ 60) | Banho e Tosa (R\$ 100)
 - Se for **Gato**: Banho (R\$ 60) | Tosa (R\$ 70) | Banho e Tosa (R\$ 110)
 - Se for **Outro**: Banho (R\$ 40) | Tosa (R\$ 50) | Banho e Tosa (R\$ 80)
2. **Retorno:** Salve o agendamento no banco e retorne o documento criado com status **201**.

3.2 Listar Todos os Agendamentos (READ)

Crie a rota `GET /agendamentos`.

Regras:

- A rota deve retornar todos os agendamentos salvos no MongoDB usando `Agendamento.find()`.
- Retorne o array de resultados com status `200`.

3.3 Buscar Agendamentos por Nome do Pet (QUERY PARAMS)

Crie a rota `GET /agendamentos/busca`.

Esta rota deve aceitar um filtro opcional via Query Params chamado `nome`:

- Exemplo de URL: `http://localhost:3000/agendamentos/busca?nome=fred`
- A busca deve retornar todos os agendamentos em que o nome do pet contenha o texto pesquisado.
- **Exemplo:** Buscar por `"fred"` deve retornar agendamentos do pet `"Frederico"`.

3.4 Update do Status do Agendamento (UPDATE)

Crie a rota `PATCH /agendamentos/:id`.

O corpo da requisição deve enviar apenas o novo status (ex: `{ "status": "Concluído" }`).

Regras:

- Busque o agendamento pelo ID e atualize o status usando `findByIdAndUpdate`.
- Se o ID não for encontrado, responda com status `404`.
- Retorne o documento atualizado com status `200`.

3.5 Remover Agendamento (DELETE)

Crie a rota `DELETE /agendamentos/:id`.

Regras:

- Remova o agendamento do banco de dados pelo ID usando `findByIdAndDelete`.
- Se não encontrar o registro, retorne status `404`.
- Se remover com sucesso, retorne status `200` com uma mensagem de confirmação.

4. Testes Esperados

Realize testes na sua API seguindo este fluxo de validação:

1. Cadastre um agendamento para um **Cão** com o serviço **Banho** (verifique se o valor gravado foi R\$ 50).
 2. Cadastre outro agendamento para um **Gato** com o serviço **Banho e Tosa** (verifique se o valor gravado foi R\$ 110).
 3. Liste todos os agendamentos cadastrados.
 4. Faça uma busca pelo termo **"fred"** e verifique se traz apenas os pets correspondentes.
 5. Atualize o status de um agendamento para **Concluído**.
 6. Delete um dos agendamentos informando o ID.
-

5. Dicas para a Implementação

- No arquivo **db.js**, use **mongoose.connect(process.env.MONGO_URI)** envolvido in um **try/catch** assíncrono.
 - No **findByIdAndUpdate**, passe **{ new: true, runValidators: true }** para que o Mongoose aplique as validações e retorne o registro já atualizado.
 - Lembre-se de usar **app.use(express.json())** no **src/server.js** para conseguir ler o corpo das requisições.
-

LionsDev • Professor Nicolas Cardoso Motta
Exercício Prático de Mongoose e MongoDB - Módulo 08